DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

# Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

**Christian Mulser**

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

# Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

# Hardwarebeschleunigtes Raytracing in dynamischen Umgebungen mittles Vulkan


| | |
|---|---|
| Author: | Christian Mulser |
| Supervisor: | Prof. Dr. Rüdiger Westermann |
| Advisor: | Prof. Dr. Rüdiger Westermann |
| Submission Date: | October 2, 2025 |

I confirm that this bachelor's thesis in informatics: games engineering is my own work and I have documented all sources and material used.

Munich, October 2, 2025 Christian Mulser

# Abstract

In recent years, real-time, hardware-accelerated ray tracing has made significant progress. Since the release of Nvidia's GeForce RTX series, real-time ray tracing has become increasingly popular in interactive applications. This thesis explores how to implement a hardware-accelerated ray tracer using the Vulkan API. We evaluate different pipelines and strategies for updating acceleration structures in dynamic scenes, analyze their performance, and discuss their advantages and drawbacks. Although the tested approaches are intentionally simple, they provide a solid foundation for understanding how ray tracing can be applied in dynamic, real-time environments, like video games.

# Contents

# 1 Introduction

For a long time, **rasterization** has been the standard for rendering 3D scenes in real-time (e.g., in video games). The basic steps of rasterization are as follows:

1. Every vertex of a 3D object gets transformed in the vertex shader.
2. The transformed triangles are projected onto a 2D pixel grid and a fragment is generated for every pixel that the triangle covers.
3. The fragment shader runs for every generated fragment. It uses the fragment's data and some global data (e.g., light sources) to determine the final color of the pixel.

Rasterization is very fast and efficient, but it also has its downsides. When determining the color of a pixel, we only have access to the data in the current fragment and some global data. The fragment is not aware of other objects in the scene, which makes effects like shadows and reflections very difficult, requiring a workaround like shadow-mapping and screen-space-reflections. In recent years, a new way to do real-time rendering slowly emerged, which, of course, is **ray tracing**.

Ray tracing works by casting rays through the screen's pixels into the 3D scene to determine the surface to be displayed. When a ray hits a surface, secondary rays, such as shadow rays, reflection rays, or refraction rays, can be cast. That is much closer to real life, where light rays bounce between objects until they reach our eyes or a camera lens, and because of that, it is much easier to generate realistic-looking images. However, it was never suitable for real-time applications in the past due to the enormous amount of processing necessary.

Even though ray tracing is relatively new in the realm of real-time graphics, the concept of ray tracing is almost as old as rasterization. In 1968, Arthur Appel explores the idea of casting a ray through a pixel to determine which surface is visible, and also the idea of casting a ray from a surface point to the light source to check if the light reaches the surface unobstructed [App68]. Later, in 1980, Turner Whitted introduced the well-known ray tracing model "Whitted-Raytracing" [Whi80]. If a ray hits a surface, additional rays are cast for reflection, refraction, and shadow. In the 1990s, ray tracing started to be used in visual effects for movies like "Terminator 2: Judgment Day" (1991) or "Jurassic Park" (1993). In the 2010s, animation studios like Pixar moved to fully path-traced rendering [Chr+18], which uses ray tracing to trace the whole path from a light source to the camera to generate photorealistic images. An example for this is Pixar's animated movie "Finding Dory" (2016) [HVH16].

In 2018, Nvidia paved the way for ray tracing in real-time applications by introducing the RTX Series. This series of graphics cards used the "Turing architecture", which introduced hardware-accelerated ray tracing with the new RT Cores. RT Cores traverse the BVH

(Bounding Volume Hierarchy) autonomously, and by accelerating traversal and ray/triangle intersection tests, they offload the SM (Streaming Multiprocessor), allowing it to handle other vertex, pixel, and compute shading work [Cor18].

Testing every ray against every triangle for intersection is not feasible and takes way too much processing power, even for non-real-time applications. A much faster way to find the triangles that are hit by a ray is to use a bounding volume hierarchy (BVH). It is a tree structure of volumes (usually AABBs), which divides the scene into smaller and smaller subsections as we go down the tree. We can prune a branch as soon as we realize that the section does not intersect with the ray and therefore avoid a lot of ray/triangle intersections.

In this thesis, we will implement hardware-accelerated ray tracing for dynamic environments using the graphics API Vulkan. We will learn how to create the raytracing pipeline, set up our shader binding table, and build and update our acceleration structures. Then, we will test different pipelines and update strategies and other parameters in dynamic scenes containing moving and deforming objects.

# 2 Related Work

## 2.1 Ray Tracing

As mentioned in the introduction Appel [App68] and Whitted [Whi80] laid the foundations for ray tracing. For a long time, GPUs were not feasible for ray tracing, and because of that, efficient ray tracing algorithms were developed for the CPU. Möller et al. [MT05] present a clean algorithm for determining whether a ray intersects a triangle. One advantage of this method is that the plane equation need not be computed on the fly nor be stored, which can amount to significant memory savings for triangle meshes. Similarly, another paper presents and efficient and robust algorithm for intersections between rays and AABBs [Wil+05], which is very useful for the bounding volume hierarchies used for acceleration structures. Wald et al. [Wal+14] describe Embree, an open source ray tracing framework for x86 CPUs, which is explicitly designed to achieve high performance in professional rendering environments in which complex geometry and incoherent ray distributions are common.

Later Nvidia introduced the Turing Architecture, which allows for hardware-accelerated raytracing. Using new hardware-based accelerators and a Hybrid Rendering approach, Turing fuses rasterization, real-time ray tracing, AI, and simulation to enable incredible realism in PC games, amazing new effects powered by neural networks, cinematic-quality interactive experiences, and fluid interactivity when creating or navigating complex 3D models [Cor18]. Huerta et al. [Hue+25] analyse these modern GPU architectures by reverse engineering modern NVIDIA GPU cores, unveiling many key aspects of their design.

## 2.2 Acceleration Structures

There has been a lot of research done on ray tracing acceleration structures. Many of these papers propose new ways of constructing a BVH and discuss the advantages and disadvangtages. One of these methods is KD-tree construction. Soupikov et al. [SK07] presents a highly parallel, linearly scalable technique of kd-tree construction for ray tracing of dynamic geometry. Another paper proposes modifications to previous kd-tree construction algorithms that significantly increase the coherence of memory accesses during construction of the kd-tree [Pop+06]. These and other optimizations are then applied to a SAH (Surface Area Heuristic)-based BVH instead of a KD-tree by Wald [Wal07]. SAH tries to minimize the surface area of the bounding volumes when splitting a parent volume.

Another type of construction is LBVH (Linear Bounding Volume Hierarchy), which uses spatial Morton codes to reduce the BVH construction problem to a simple sorting problem [Lau+09]. There is also another variant of the algorithm called HLBVH (Hierarchical LBVH).

It is a novel hierarchical algorithm that reduces substantially both the amount of computations and the memory traffic required to build a Linear Bounding Volume Hierarchy (LBVH), while still exposing a data-parallel structure [PL10]. Karras et al. [KA13] propose a new massively parallel algorithm for constructing high-quality bounding volume hierarchies (BVHs) for ray tracing, which is based on modifying an existing BVH to improve its quality, and executes in linear time.

A big topic in dynamic environments is updating the acceleration structure. Wald et al. [WBS07] present three algorithmic changes to the typical BVH algorithm to benefit dynamic scenes. First, the BVH is built using a variant of the surface area heuristic conventionally used to build kd-trees. Second, the topology of the BVH is not changed over time so that only the bounding volumes need to be refit from frame-to-frame. Third, and most importantly, packets of rays are traced together through the BVH using a novel integrated packet-frustum traversal scheme. Later, Kopta et al. [Kop+12] describe a method to efficiently extend refitting for animated scenes with tree rotations, a technique previously proposed for off-line improvement of BVH quality for static scenes.

## 2.3 Skinning Techniques

Since our application uses skeletal animation, we also need to consider different skinning methods. In our application, we use LBS (Linear Blend Skinning), which is a very popular skinning technique due to its simplicity and efficiency. However, LBS does have some issues (e.g., collapsing-joint artifacts), and some research addresses these issues and proposes better alternatives.

Kavan et al. [KŽ05] introduce a new method called SBS (Spherical Blend Skinning). It uses quaternions to interpolate rotations instead of linearly blending transformed vertex positions. A significant advantage of this method is that the input data for this algorithm is the same as for LBS. Another paper presents a novel GPU-friendly skinning algorithm based on dual quaternions [Kav+07]. This approach also solves the artifacts of linear blend skinning at minimal additional cost.

## 2.4 Summary and Research Gap

We have now reviewed a lot of research regarding ray tracing in dynamic environments. However, the presented research on acceleration structures does not include scenes that use a scene graph or skeletal animation. The test scenes in these papers usually use a single acceleration structure for the whole scene, which gets updated every frame.

Vulkan uses a two-level hierarchy for its acceleration structure, which allows us only to update parts of the scene's acceleration structure. With this thesis, we want to explore real-time raytracing in dynamic scenes using a scene graph and skeletal animation, not only because we want to see how a scene graph takes advantage of Vulkan's two-level acceleration structure, but also because scene graphs and skeletal animation are very common in real-time applications, like video games.

# 3 Technologies

## 3.1 System Specifications

| | |
|---|---|
| **Operating System** | Ubuntu 24.04.2 LTS |
| **CPU** | Intel(R) Xeon(R) W-2235 CPU @ 3.80GHz |
| **GPU** | NVIDIA GeForce RTX 3090 |

Table 3.1: System Specifications

## 3.2 Build Tools

The project uses **Premake 5**, which does not build the project directly. It is a command-line utility that reads a scripted definition of a software project and generates project files for toolsets like Visual Studio, Xcode, or GNU Make. It is a simpler alternative to CMake.

We use Premake to create **Makefiles**, which then ultimately compile our project.

## 3.3 Graphics API

As our project's graphics API, we chose **Vulkan 1.2**. Vulkan is a modern platform-agnostic API that handles GPU operations. It is the successor to OpenGL, which was discontinued in 2016. Vulkan allows for much more fine-grained control over the GPU, supports multi-threading, and most importantly for this project, raytracing, through extensions like `VK_KHR_ray_tracing_pipeline` and `VK_KHR_acceleration_structure`. Since these extensions were introduced in Vulkan 1.2, that is the version used in the project.

## 3.4 Programming Language

At the start of the project, Rust was considered for the project's programming language. Rust is a new, memory-safe, low-level programming language and a modern alternative to C and C++. While Rust has many advantages over C++, we chose **C++** in the end. The main reason is that the Vulkan API was designed for C/C++. There are a lot of great Vulkan libraries for Rust, like the popular "Ash" crate, which is generated from "vk.xml". However, the interface differs from the classic C API in some parts. That could be problematic, considering that nearly all the few resources relating to raytracing in Vulkan use C/C++. That could have made it challenging to translate the raytracing code into Rust later down the line, so C++ was the safer choice.

## 3.5 File Formats

### 3.5.1 glTF

The dynamic 3D scenes that the project uses are stored in the **glTF-2.0** [Khr21] file format. It stores its models in a scene graph and supports skeletal animation and morph animations. By default, it stores the scene data in the popular JSON format. Additionally, buffers are stored in binary BIN files, while textures are stored in image files (e.g., PNG, JPEG, . . . ). Since the file format was developed by Khronos Group, which is also behind Vulkan, the data inside the buffers is laid out in a way that makes it easy to use in Vulkan.

### 3.5.2 CSV

**CSV** [Sha05] stands for Comma-Separated Values and is used to store tabular data. It is a text-based format and, therefore, very easy to use. Each line of text inside a CSV file represents a row, and the values of that row are separated by commas. We use it to store our benchmarks in our projects. We can later import a CSV file into an application like Excel to create graphs and analyse our gathered data.

## 3.6 Libraries

### 3.6.1 volk

**volk** [zeu] is a meta-loader for Vulkan. It allows you to dynamically load entry points required to use Vulkan without linking to vulkan-1.dll or statically linking Vulkan loader. Additionally, volk simplifies the use of Vulkan extensions by automatically loading all associated entry points.

### 3.6.2 VMA (Vulkan Memory Allocator)

Memory Management in Vulkan is quite tricky. For example, when a buffer or image is created in Vulkan, no memory is associated with it. The programmer must manually allocate a block of memory with a specific type from a particular memory heap. The properties of these memory heaps depend on the underlying physical device and need to be queried first. Additionally, creating a new memory allocation for each resource may be suboptimal, since the number of allocations can be very limited and many allocations may negatively impact performance. **VMA (Vulkan Memory Allocator)** [GPU] simplifies, automates, and optimizes memory management in Vulkan.

### 3.6.3 vk-bootstrap

Initializing Vulkan involves a lot of boilerplate code, which will be basically the same in every Vulkan application. **vk-bootstrap** [Lun] can reduce the amount of boilerplate code in a project.

This library simplifies the process of:

- Instance creation
- Physical Device selection
- Device creation
- Queue aquisition
- Swapchain creation

### 3.6.4 cgltf

**cgltf** is used to load a glTF file in the project. It parses the glTF file, which is stored in JSON. It would also have been possible to read the JSON directly, but cgltf validates the file, loads the required buffers from their BIN files into memory, and creates a `cgltf_data` struct, which is a lot more comfortable to use. Our project does not use this structure directly, but instead uses it to generate a `Scene` object.

### 3.6.5 CLI11

**CLI11** is a header-only library written in modern C++ designed to parse your application's command-line options. It is simple, feature-rich, and safe. CLI11 allows you to define flags, options, commands, and subcommands. You can also add restrictions to parameters, like only allowing positive numbers or ensuring that a parameter is an existing path.

# 4 Implementation

## 4.1 Vulkan Abstraction

### 4.1.1 Vulkan Handles and Move Semantics

As mentioned in the Methods and Technology chapter, we initially considered Rust as the programming language for our application, because it is modern and memory safe. One of Rust's core features is ownership, which means that each value or object can only be owned and managed by a single variable. When that variable goes out of scope, it is solely responsible for cleaning up the object. If the value is assigned to another variable, the new variable becomes the owner (i.e., the value is moved). Ownership would be a handy feature for managing Vulkan objects. Vulkan objects should not be copied on assignment but only moved, and when the object is no longer needed (i.e., the owner goes out of scope), it should be automatically destroyed.

While C++ does not support ownership directly, it does provide move semantics, which allow us to emulate it. The ownership of Vulkan handles is managed by the `Handle<T>` class. To achieve this, the class declares a default constructor, a move constructor, a destructor, and overrides the move assignment operator. It is also important that the copy constructor and copy assignment operator are deleted. The reason is that copying a Vulkan object is usually very expensive or not even possible, and it is almost never necessary. When we actually want to copy an object, we need to explicitly create a new one before copying the contents of the other object into it (for example, using `Buffer::copy_into`).

### 4.1.2 Shared Pointers

Since C++11, the language supports smart pointers, which make memory management safer and less complicated. One of these smart pointers is `std::shared_ptr<T>` (or our alias `ptr::Shared<T>`). This pointer type allocates an object that can then be shared between multiple owners. The object remains alive as long as at least one owner exists, and the last owner that goes out of scope cleans it up. This structure works very well with our previously mentioned `Handle<T>`. A Vulkan object can only have one owner, which is a harsh limitation and often leads to problems. If we wrap the object in a shared pointer, it can be shared across multiple owners.

### 4.1.3 The Builder Pattern

As mentioned before, we want the creation of Vulkan objects to be very explicit. The first method you might think of for creating objects in C++ is constructors, but they are problematic for a few reasons. The first issue is that constructors are often called implicitly (e.g., the default constructor is called when declaring a variable). Default construction only makes sense for very few Vulkan objects (such as a `VkFence`), and we do not want a Vulkan object to be created when a default constructor is implicitly (or explicitly) called. For this reason, the default constructor simply does not create a Vulkan object and leaves the Vulkan handle at `VK_NULL_HANDLE`. If another constructor actually creates an object, the behavior of constructors becomes inconsistent with Vulkan object creation. In addition, Vulkan objects are almost always created using a `Vk*CreateInfo` struct, which may require many parameters.

To solve these issues, we implemented a `*Builder` class for most Vulkan objects. Some exceptions are `Fence` and `ImageView`, which are instead created with the static methods `Fence::create` and `ImageView::create_from`, respectively. That makes it very clear when a Vulkan object is created, since it can only be created using a `*Builder` or a static method (such as `Fence::create`), and especially because it can never be created by a constructor. For this reason, Vulkan objects never have a constructor other than the default constructor.

```
// declare the buffer (handle is VK_NULL_HANDLE)
Buffer buffer;

// creating the buffer
buffer = BufferBuilder()
    .add_usage(VK_BUFFER_USAGE_TRANSFER_DST_BIT)
    .add_usage(VK_BUFFER_USAGE_VERTEX_BUFFER_BIT)
    .memory_usage(VMA_MEMORY_USAGE_GPU_ONLY)
    .size(128)
    .build();
```

Figure 4.1: Sample-code of the builder pattern for a buffer

### 4.1.4 Basic Vulkan Object Structure

All of the Vulkan objects follow a basic structure, which can be seen in the code sample of Figure 4.2.

```cpp
class Foo : public utils::Move, public ptr::ToShared<Foo> {
    friend class FooBuilder;
private:
    Handle<VkFoo> foo_handle{};
    // Additional member variables

public:
    Foo() = default;

    inline VkFoo handle() const { return foo_handle; }
    // Additional getter functions

    // Additional functions
};

class FooBuilder {
public:
    using Ref = FooBuilder&;
private:
    // build parameters, e.g. VkFormat _format;

public:
    FooBuilder() = default;

    // parameter setter functions, e.g. Ref format(VkFormat format);

    Foo build() const;
}
```

Figure 4.2: The basic structure of every class representing a Vulkan object

It is important to note that `Foo` is just a placeholder for an actual Vulkan object (such as `Buffer`, `Image`, or `Pipeline`). Usually, the name of the class is simply the name of the Vulkan handle without the `Vk` prefix (e.g., `VkBuffer` $\implies$ `Buffer`), but there are some exceptions (e.g., a `Shader` class holds a `VkShaderModule` handle). As we can see, the Vulkan handle is wrapped in the `Handle<T>` class, which ensures correct move semantics. The member function `handle` allows us to access the raw Vulkan handle.

The class inherits from `utils::Move` and `ptr::ToShared<Foo>`. The `utils::Move` class implements the default move constructor and move assignment operator, and deletes the copy constructor and copy assignment operator. The `ptr::ToShared<Foo>` class provides the `to_shared` function, which is just syntax sugar for moving the object into a shared pointer.

Additional member variables in the class are often used to store metadata (e.g., the format and extent of an image) or to keep other objects alive in a shared pointer that are referenced by the underlying object (e.g., a `RenderPass` is referenced by a `Pipeline`).

There may also be additional functions that use or manipulate the underlying object. Often, these functions have a `cmd_` prefix, which means that they are wrappers around one or multiple commands (e.g., `Buffer::cmd_copy_into` is a wrapper for the Vulkan function `vkCmdCopyBuffer`).

Even though not every Vulkan object uses a `*Builder` class, it is shown in the sample code since most Vulkan objects have one.

### 4.1.5 The Vulkan Context

Before we can do any work in Vulkan, a lot of setup is required, which usually includes the following steps:

1. Create a `VkInstance`.
2. Select a `VkPhysicalDevice` that supports the extensions, features, and capabilities required by the project.
3. Create a `VkDevice` from the selected `VkPhysicalDevice`.
4. When creating the `VkDevice`, enable `VkQueue`s from a particular queue family so that command buffers can be submitted later.

These Vulkan objects are accessed frequently throughout the project. For example, we need to pass the `VkDevice` as a parameter when creating or destroying most Vulkan objects, and a command buffer that executes a sequence of Vulkan commands needs to be submitted to a specific `VkQueue`. Keeping track of all these Vulkan objects is tedious, so we introduced the Vulkan context. It is represented by the `Context` class, which stores these Vulkan objects, provides access to them anywhere, and reduces the effort of setting up Vulkan.

The Vulkan context contains the following objects:

- the `VkInstance`
- the `VkPhysicalDevice` and `VkDevice`
- an optional `GLFWwindow*` from which a `VkSurfaceKHR` is created and also stored (this is necessary for rendering to a window, but it is not used in our main application since it is headless)
- the `CommandManager` (manages queues and command pools)
- the `VmaAllocator` (used when allocating memory)

The class is designed similarly to a singleton, but it differs in some parts. A singleton usually creates its instance at the start of the application or when it is used for the first time (lazy instantiation). However, `Context` cannot be implicitly instantiated this way because it needs parameters stored in a `ContextInfo` when being created. For this reason, we need to explicitly create the Vulkan context using `Context::create` with the desired `ContextInfo` before accessing it for the first time using `Context::instance`, or we will get a `nullptr`. While all other objects in our project are cleaned up automatically when going out of scope, the Vulkan context must be destroyed explicitly using `Context::destroy()` when it is no longer needed. If we now want to get the current `VkDevice`, we can call `Context::instance()->get_device()`.

## 4.2 Raytracing-related Objects

In this section we will take a closer look at all the objects related to ray tracing and our abstractions of them.

### 4.2.1 Acceleration Structures

We are going to start with the acceleration structures. Acceleration structures are data structures used by the implementation to efficiently manage scene geometry as it is traversed during a ray tracing query [Khr25]. The acceleration structure has two levels:

- **Bottom Level Acceleration Structure (BLAS)**

  The lower level, the bottom-level acceleration structure, contains the triangles or axis-aligned bounding boxes (AABBs) of custom geometry that make up the scene [Koc+20]. The BLAS usually corresponds to a mesh that is used in a scene.

- **Top Level Acceleration Structure (TLAS)**

  The upper level, the top-level acceleration structure, contains references to a set of bottom-level acceleration structures, each reference including shading and transform information for that reference [Koc+20]. The TLAS represents a scene filled with objects, each of which can have a different transformation and a mesh.

The relationship between TLAS and BLAS can be seen in Figure 4.3.

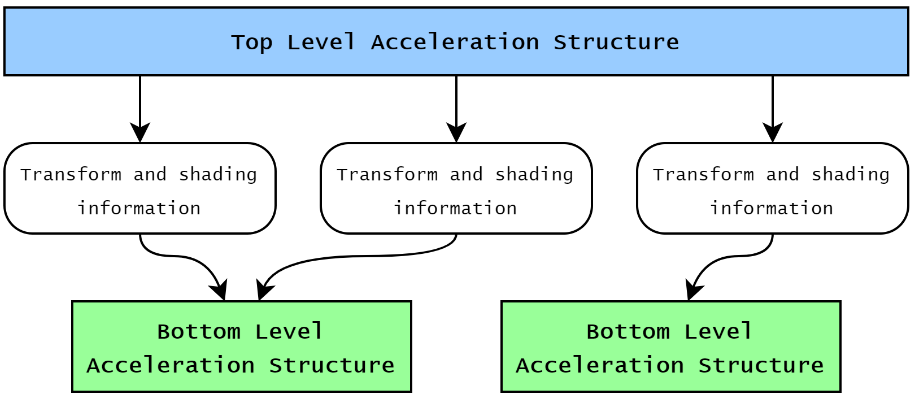



Figure 4.3: Acceleration structure

Vulkan represents both types using the `VkAccelerationStructureKHR` handle. This handle only contains some metadata; the actual data of the acceleration structure (e.g., the BVH) is stored in a `VkBuffer`. Before building an acceleration structure, we need to reference the data used for building in a `VkAccelerationStructureGeometryKHR`. For a BLAS, this is typically a list of triangles (other primitives are also possible), and for a TLAS, it is a list of instances containing transformation and shading information, as well as a handle to a BLAS. Once the `VkAccelerationStructureGeometryKHR` is prepared, we can query `VkAccelerationStructureBuildSizesInfoKHR`, which contains `accelerationStructureSize` (the buffer size for the acceleration structure) and `updateScratchSize` and `buildScratchSize` (the sizes of temporary scratch buffers used during building or updating). Finally, we can build the acceleration structure using `vkCmdBuildAccelerationStructuresKHR`.

We encapsulate a BLAS and TLAS in the classes `Blas` and `Tlas`, respectively. They both have the following member variables:

- `blas` / `tlas`: The `Handle<VkAccelerationStructureKHR>`.
- `buffer`: The buffer that stores the acceleration structure data.
- `build_scratch_buffer`: The scratch buffer used during building.
- `update_scratch_buffer`: The scratch buffer used during updating/refitting.

These classes also have a builder (see subsection 4.1.3 and subsection 4.1.4), which expects a list of `BlasGeometry`s when building a `Blas` and a list of `TlasInstance`s when building a `Tlas`. A `BlasGeometry` describes the vertex buffer and optional index buffer used for a triangle mesh, while a `TlasInstance` stores a reference to a BLAS in a `ptr::Shared<Blas>` together with a transformation. The builder has a `dynamic` property, which must be set to `true` if the acceleration structure will be refitted later. It also has a `fast_build` property, which, if set to `true`, will prefer a fast build over fast tracing by setting the flag `PREFER_FAST_BUILD` instead of `PREFER_FAST_TRACE`.

There are two ways to update an acceleration structure:

- `rebuild`

  Recreates the BVH from scratch. It performs the same steps as the builder's `build` function, but uses the already created handle and buffers without creating them again.

- `refit`

  Adjusts only the bounds of the AABBs in the existing BVH according to new vertex positions, which is faster than rebuilding from scratch but may result in a suboptimal BVH, which can increase ray tracing time.

### 4.2.2 Raytracing Pipeline

To issue ray tracing commands, we need a ray tracing pipeline. Figure 4.4 shows a flow chart of the pipeline.

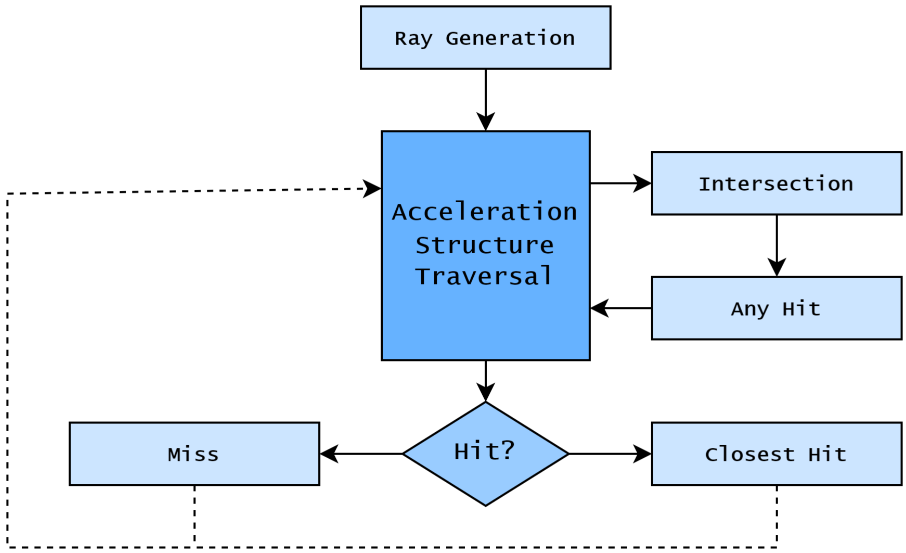



Figure 4.4: Ray tracing pipeline

A Vulkan raytracing pipeline organizes shaders into shader groups. In our application, we use two types:

- **General Shader Group**

  A general shader group contains a single general shader that can be called independently of any ray/triangle intersection. This type is used in the ray generation and miss stages.

- **Triangle Hit Shader Group**

  A triangle hit shader group contains an any-hit shader and a closest-hit shader, executed when a ray intersects a triangle. The any-hit shader runs for every intersection along the ray, while the closest-hit shader runs only for the first intersection.

The class `ShaderGroup` implements shader groups in our application. Each `ShaderGroup` has a unique identifier accessible via the function `id`. It stores either a `ShaderGroupGeneral` or a `ShaderGroupHit` struct in a `std::variant`. `ShaderGroupGeneral` contains the `general` shader, while `ShaderGroupHit` contains `closest_hit` and `any_hit` shaders.

The raytracing pipeline is encapsulated in the class `RtxPipeline`, which is created from a list of `ShaderGroup`s. The `ShaderGroup` objects are only descriptions; the pipeline allocates the actual shader groups and stores a list of shader group handles. `RtxPipeline` also maintains a map from each `ShaderGroup` ID to the index of the shader group handle array for easy access.

The pipeline also needs a pipeline layout, given by a `PipelineLayout` object, and a maximum recursion depth (`max_ray_recursion_depth`) for rays.

### 4.2.3 Shader Binding Table

Once the ray tracing pipeline is ready we need to create a shader binding table (SBT), which selects the correct shader group in every stage of the raytracing pipeline.

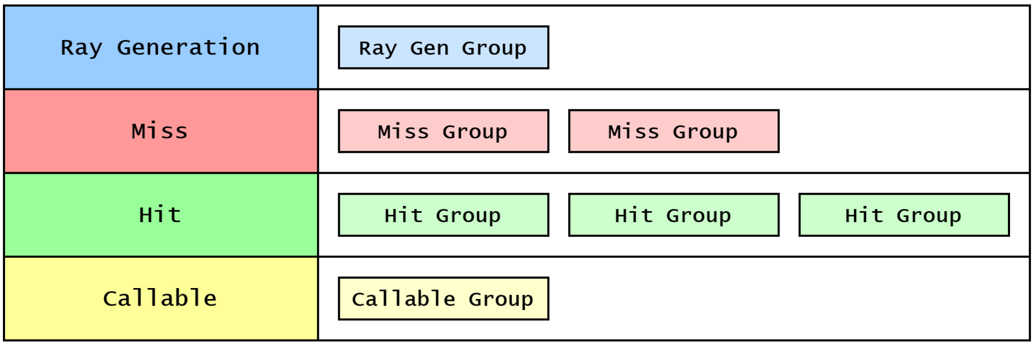



Figure 4.5: Shader binding table

As seen in Figure 4.5, the SBT has four rows:

- **Ray Generation**: Shader group used for ray generation.
- **Miss**: Shader groups used when a ray misses.
- **Hit**: Shader groups used when a ray intersects at least one object.
- **Callable**: Arbitrary shaders callable by other shaders, not tied to a specific ray traversal event (not used in our application).

The SBT is stored in a `VkBuffer`. A `VkStridedDeviceAddressRegionKHR`, which references a buffer region and specifies the stride between elements, represents a row in the SBT.

When casting a ray with `traceRayEXT`, we can specify a miss offset and a hit offset, indicating which shader group to use. Each TLAS instance can also define an additional offset for the hit shader group, added to the previous offset.

The SBT is implemented in the `SBT` class. Since the SBT strongly depends on the raytracing pipeline, it does not use a builder. Instead, it is built using the `build_sbt` function in `RtxPipeline`, with a `SBTInfo` struct as a parameter. `SBTInfo` contains:

- `ray_gen_group`: The `ShaderGroup` used for ray generation.
- `miss_groups`: A list of `ShaderGroup`s, which can be used when a ray misses.
- `hit_groups`: A list of `ShaderGroup`s, which can be used when a ray hits.

The `ShaderGroup`s in `SBTInfo` must be a subset of the shader groups used to create the pipeline. When building the SBT, we first retrieve the array of shader group handles using `vkGetRayTracingShaderGroupHandlesKHR`. Using the ID-to-index map, we get the handle for each shader group and store them in the SBT buffer: ray generation first, followed by miss shaders, then hit shaders. Finally, the `VkStridedDeviceAddressRegionKHR` is set for each row, and the SBT is ready for use in `vkCmdTraceRaysKHR`.

The code sample in Figure 4.6 shows the creation of a ray tracing pipeline and SBT:

```
// creating shader groups from shaders
ShaderGroup ray_gen_group = ShaderGroup::create_general(ray_gen_shader);
ShaderGroup miss_group = ShaderGroup::create_general(miss_shader);
...
ShaderGroup hit_group = ShaderGroup::create_hit_closest(closest_hit_shader);
...

//creating the ray tracing pipeline
RtxPipeline pipeline = RtxPipelineBuilder()
    .add_shader_group(ray_gen_group)
    .add_shader_group(miss_group)
    .add_shader_group(hit_group)
        ...
    .layout(pipeline_layout)
    .max_ray_recursion_depth(1)
    .build();

//createing the SBT
SBT sbt = pipeline.build_sbt(SBTInfo()
    .ray_gen_group(ray_gen_group) // ray gen row
    .miss_groups({ miss_group, ... }) // miss row
    .hit_groups({ hit_group, ... })); // hit row
```

Figure 4.6: Sample-code to create ray tracing pipeline and SBT

## 4.3 Scene

In this section we are looking at the structure and behaviour of our implementation of a scene. In order to understand, why we structured our scenes the way we did, we first need to take a look at the glTF file format, from which our scenes are loaded from.

### 4.3.1 glTF Structure

A glTF file stores its data in the JSON format. There are different types of objects in glTF and each of them is stored in an array. An objects can reference an other object by its index in the respective array. The objects we are interested in are:

## 4.4 Scene

In this section, we are looking at the structure and behaviour of our scene implementation. In order to understand why we structured our scenes the way we did, we first need to take a look at the glTF file format from which our scenes are loaded.

### 4.4.1 glTF Structure

A glTF file stores its data in the JSON format. There are different types of objects in glTF, each stored in an array. An object can reference another object by its index in the respective array. The objects we are interested in are:

- **Buffer, Buffer view and Accessor**

  These three objects work together closely. A buffer stores a block of binary data, usually contained in a separate BIN file, and a buffer view describes a specific part of a buffer. A buffer view contains a reference to a buffer and an offset, length, and optional stride. A buffer view sees the data as just raw bytes. To associate some meaning to the data, we use an accessor. It references a buffer view at a particular offset and contains the number of elements, as well as the type (e.g., scalar, vec3, mat4) and component type (e.g., float, unsigned int) of the elements. Accessors can be used for things like vertex attributes and joint matrices.

- **Mesh**

  A mesh stores a static 3D model. It contains a material and an array of primitives. Each primitive has a mode (e.g., points, lines, triangles), an optional accessor for indices, and a set of accessors for the different vertex attributes (e.g., positions, normals, uv coordinates, vertex colors, joint weights).

- **Skin**

  A skin is used for skeletal animation of a mesh. It contains an array of nodes that are used as the skeleton's joints and an accessor for the inverse bind matrices of the joints.

- **Node**

  This represents a node in the scene graph. A node always has a transform, which is either represented by a 4x4 matrix or a scale (vec3), rotation (quaternion), and translation (vec3). A node can have an array of child nodes, which inherit the transform of the parent node. In other words, a node's transform is applied in the parent node's coordinate system.

  A node can optionally also contain a mesh and, if a mesh is present, also a skin. The skin can deform the node's mesh during an animation.

- **Scene**

  A scene contains an array of root nodes. These nodes and all of their children are part of that scene.

- **Animation**

  An animation has an array of channels and an array of samplers. A channel describes which property is being sampled during the animation. It contains the affected node and a path for the node's affected property (e.g., "transform", "scale", "rotation"). It also holds a reference to one of the animation samplers.

  A sampler calculates a property's value at a certain point in time from a set of key frames. It contains two accessors, one for the time of the key frames and one for the value of the key frames. Additionally, a sampler contains the interpolation type that should be used to sample a value.

There are also some other objects (e.g., materials, textures, and cameras), but since they are not relevant to our application, we don't explain them here.

### 4.4.2 Scene Structure

We will now take a look at the structure of our scene implementation. For this we will look at the C++ classes we used to implement the different glTF objects.

#### The `Scene` class

The `Scene` class represents a scene loaded from a glTF file. It contains an array of nodes, an array of animations, and an array of skins. Additionally, it stores the TLAS used for ray traversal and the buffers used by the scene in a `SceneBuffers` struct.

A scene can be loaded using the static method `Scene::load`, which takes the path to the glTF file as a parameter. The class also provides `Scene::iter`, which returns an iterator over all nodes in the scene. The function `Scene::build_acceleration_structures` creates our acceleration structures and must be called when using the scene for ray tracing. Other scene functions are explained in the following subsection on updating a scene (see subsection 4.4.3).

**The `Mesh` class**

The `Mesh` class represents a glTF mesh. It stores an array of `Primitive`s. Each `Primitive` contains three sub-buffers: `positions` for vertex positions, `indices` for vertex indices, and `joint_weights` for joint weights used in skinning. The positions and indices store sequences of `vec3`s and `unsigned int`s, respectively. Joint weights are stored as a 2D array of `JointWeight` structs, where each `JointWeight` contains the index of a node in the skeleton and its corresponding weight. All joint weights for a vertex are stored sequentially, as shown in Figure 4.7.

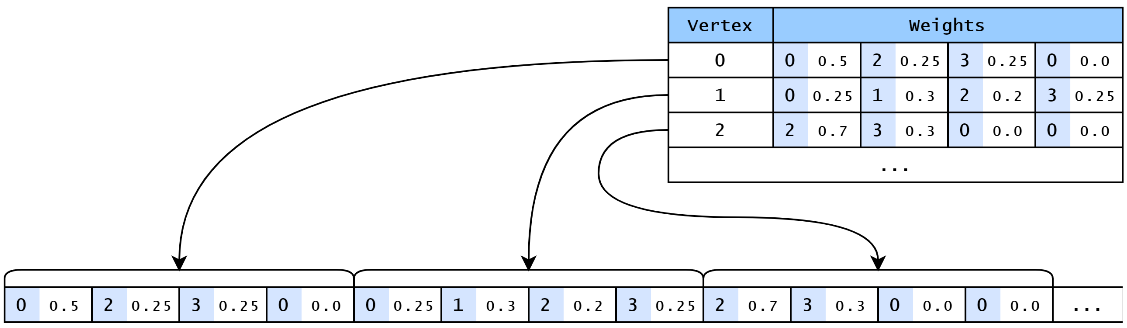



Figure 4.7: Joint weight layout

Each `Primitive` also contains `positions_cpu` and `joint_weights_cpu`, which store the same data in `std::vector`s for easier CPU access during skinning.

A `Mesh` can optionally contain a `Blas`, created for the static mesh, i.e., a node uses the mesh without a skin.

**The `Skin` class**

The `Skin` class closely mirrors the glTF skin object. It stores an array of `Node`s representing the joints in the skeleton and an array of inverse bind matrices, with one entry per joint.

**The `Node` class**

The `Node` class represents a glTF node. The node transform can be stored as a matrix or as separate translation, rotation, and scale values. To model this, we use the `RawTransform` struct, which contains translation, rotation, and scale, and the `NodeTransform` struct, which stores the local transform (see Figure 4.8).

```
struct NodeTransform {
    bool raw;
    union {
        glm::mat4 matrix;
        RawTransform raw;
    } transform;
}
```

Figure 4.8: Structure used to store local transform of a node

The union `transform` stores either a `glm::mat4` or a `RawTransform`. The boolean `raw` determines how the union should be interpreted: if `raw` is `true`, it contains a raw transform; otherwise, it contains a matrix. The node also stores a global transform as a 4x4 matrix, which is only valid after calling `Node::update_global_transform`.

A node stores an array of child nodes and optionally a `Mesh`. If a mesh is present, the node stores a `Blas` and must generate a `TlasInstance` referencing its `Blas` when building the acceleration structure. After the first build, the node stores the index of its `TlasInstance`, used for updates. If the node has no `Skin`, its `Blas` is the same as the mesh's `Blas`.

If the node contains a `Skin`, it stores an array of sub-buffers for dynamic positions, which store vertex positions after skinning. This array contains one element for every primitive in the node's mesh. Each sub-buffer matches the size of the corresponding static positions sub-buffer. In this case, the node's `Blas` is dynamic and updated with the node's `dynamic_positions`, rather than referencing the mesh's static `Blas`.

#### The `Animation` class

The `Animation` class represents a glTF animation. It contains an array of `AnimationSampler`s, each referencing the node it modifies and implementing a virtual `apply_for` function. This function samples a value at a given time and applies it to a node's transform. Three subclasses of `AnimationSampler` override `apply_for`:

- `TranslationSampler`: samples a 3D vector and sets the node's translation.
- `RotationSampler`: samples a quaternion and sets the node's rotation.
- `ScaleSampler`: samples a 3D vector and sets the node's scale.

The animation itself also implements `apply_for`, iterating over all channels and applying them at a given time.

#### Buffers and Sub-Buffers

glTF uses buffers, buffer views, and accessors. We do not use the glTF buffers directly; instead, we reorganize the data for convenience. Accessors may point to interleaved or sparse data, and component types may differ. To obtain sequential, fixed-type arrays, we use the cgltf functions `cgltf_accessor_unpack_floats` for floats and `cgltf_accessor_unpack_indices` for unsigned integers. These functions read an accessor's elements and store them in a sequence of the desired type in a target buffer.

We create separate buffers for vertex indices, joint weights, static positions, and dynamic positions. Attributes reference a sub-section of a buffer using a sub-buffer. Each sub-buffer holds a reference to the buffer (`ptr::Shared<Buffer>`), an offset, and a length. The element type is implied by the variable storing the sub-buffer (e.g., `positions` in a `Mesh` stores `vec3`s).

Figure 4.9 illustrates sub-buffer usage:

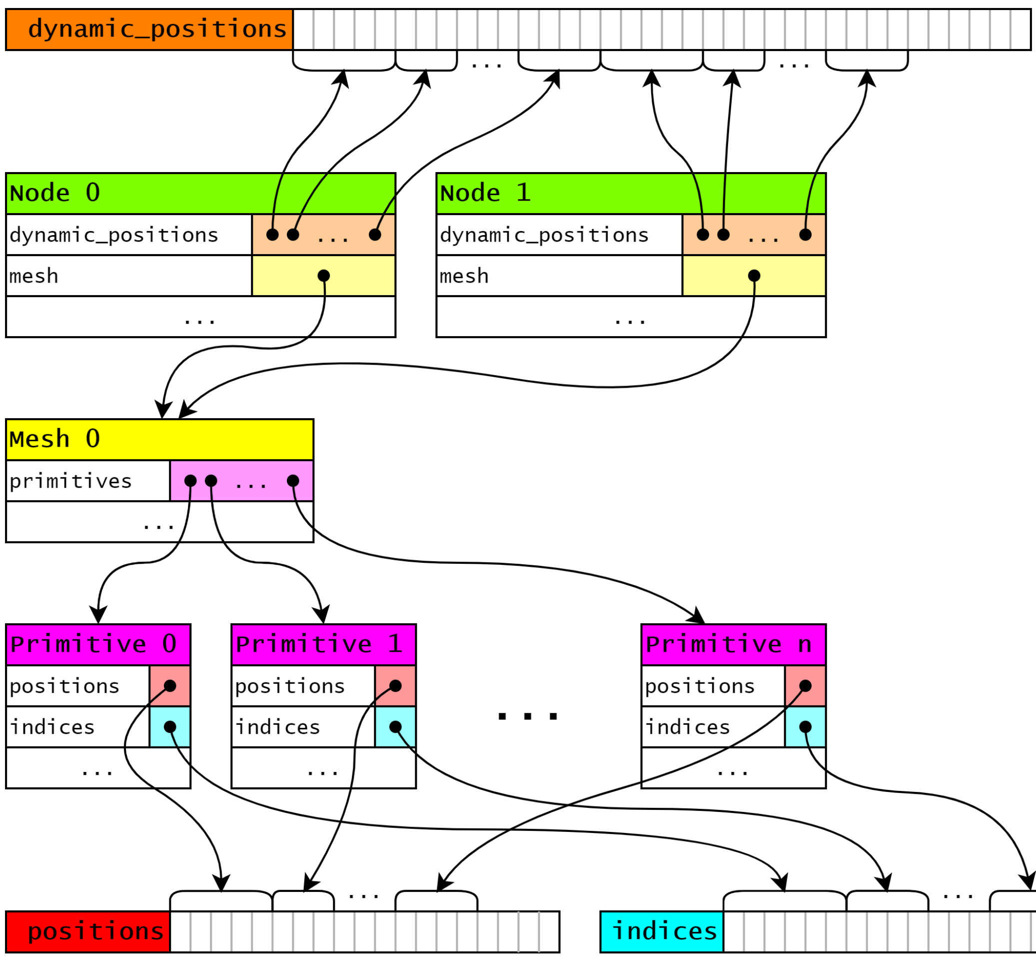



Figure 4.9: Usage of sub-buffers in a scene

The example in Figure 4.9 shows two nodes (`Node 0` and `Node 1`), which reference the same mesh (`Mesh 0`) and therefore share static positions and indices. However, since each node may use a different skin, each stores its own `dynamic_positions` sub-buffer matching the size of the static positions. For simplicity, joint weights are not shown in the example.

### 4.4.3 Updating a Scene

Since scenes are dynamic, they must be updated after calling `Animation::apply_for`. Updating occurs in three steps:

1. Update global transforms for all nodes.
2. Skin all animated models.
3. Update the acceleration structure.

The order is important, but we separate the steps to allow alternative implementations (e.g., CPU or GPU skinning, rebuilding or refitting the acceleration structure).

#### Updating Global Transform

`Node::update_global_transform` updates a node's global transform. It takes an optional `glm::mat4` parent transform, defaulting to the identity matrix.

Pseudo-code:

**Algorithm 1** Pseudo code for updating the global transform

```
function UPDATE_GLOBAL_TRANSFORM(node, parent_transform)
    node.global_transform ← parent_transform × node.transform
    for all child in node.children do
        update_global_transform(child, global_transform)
    end for
end function
```

The global transform is computed by pre-multiplying the parent transform with the node's local transform. This function is called recursively for all child nodes.

#### Skinning

Skinning applies skeleton deformation to a mesh. The first step in the skinning process is calculating the joint matrices:

**Algorithm 2** Calculating joint matrices

```
for i ← 0 to joints.count - 1 do
    joint_matrices[i] = inverse(node.global_transform)
      × joints[i].global_transform
      × inverse_bind_matrices[i]
end for
```

The mesh is then skinned:

**Algorithm 3** Skinning a model

```
for j ← 0 to (positions.count −1) do
    matrix ← 0_{4×4}
    for all joint_weight in joint_weights[i] do
        matrix ← matrix + joint_weight.weight × joint_matrices[joint_weight.index]
    end for
    position ← matrix × position
    dynamic_positions[i] ← matrix × positions[i]
end for
```

This algorithm uses **Linear Blend Skinning**, which is the standard skinning technique to its simplicity and efficiency [Jac+12]. For each vertex, we compute a weighted sum of joint matrices and apply it to the vertex position.

Our application offers two implementations:

- **CPU skinning**

  Reads positions from a primitives `positions` sub-buffer, runs the algorithm on the CPU, and writes results to the respective `dynamic_positions`.

- **GPU skinning**

  Uses a vertex shader to process each vertex and store results in a storage buffer. Rasterization is disabled, stopping the pipeline after the vertex shader. A compute shader could also be used, but this approach uses the rasterization pipeline already implemented in our application.

### Updating the acceleration structure

**Algorithm 4** Updating acceleration structure

```
function UPDATE_ACCELERATION_STRUCTURE(scene)
    node_iterator = get_node_iterator(scene)
    tlas_instances = get_instances(scene.tlas)
    for all node in node_iterator do
        if has_blas(node) then
            if is_dynamic(node.blas) then
                update(node.blas)
            end if
            update_tlas_instance(tlas, node)
        end if
    end for
    update(scene.tlas)
end function
```

There is no single `Scene::update_acceleration_structures` function. Instead, our application offers two functions:

- `Scene::rebuild_acceleration_structures`
- `Scene::refit_acceleration_structures`

Both follow the same algorithm shown in Algorithm 4; the only difference is whether the acceleration structures are rebuilt from scratch or only refitted.

## 4.5 Application Overview

## 4.6 Application Overview

In this section, we describe the functionality of our application. The application can be started with several options:

- **Test Scene**: The scene to run, selected from a set of test scenes.
- **Pipeline**: The rendering pipeline to use (see subsection 4.6.1).
- **Frame Count**: The number of frames to render.
- **Resolution**: The resolution of the frames.
- **SPP (Samples per Pixel)**: The number of rays cast per pixel (see subsection 4.6.2).
- **Delta Time**: The amount of animation time to advance between frames.
- **Rebuild Frequency**: The number of frames after which the scene is rebuilt.
- **Fast Build vs Fast Trace**: Whether to prioritize fast builds or fast ray tracing.

At startup, the application creates the specified ray tracing pipeline and loads the selected test scenes. It then applies the scene's animation for the current time, updates the nodes' global transforms, performs mesh skinning, and updates the scene's acceleration structures. After that, rays are traced, and the animation time is advanced by the delta time. These steps are repeated for the specified number of frames.

The application tracks the duration of acceleration structure updates, ray tracing, and their combined time for each frame. At the end of the run, it saves the tracked times along with their averages in a CSV file.

### 4.6.1 Pipelines

There are three possible pipelines:

- **Basic Pipeline**

  This is the simplest pipeline, casting only primary rays. If a primary ray hits an object, the pixel is colored white; otherwise, it is colored black.

- **Standard Pipeline**

  This pipeline extends the basic pipeline. Its shaders have access to a directional light source and the scene's geometry, allowing for simple diffuse shading when a ray intersects an object.

- **Shadow Pipeline**

  This pipeline extends on the standard pipeline and uses secondary rays. When a primary ray hits a surface, a shadow ray is cast from the intersection point toward the light source. The ray checks if the path to the light is unobstructed before shading the sample.

### 4.6.2 Multi-Sampling

The application supports multi-sampling, which is a common anti-aliasing technique. We also use it to increase the number of rays in our test scenes without generating enormous images.

We use a straightforward method, where each pixel is divided into an $n \times n$ grid. This only works if the number of samples per pixel is a perfect square. One ray is cast through the center of each grid cell, and the final color of the pixel is determined by averaging all of the pixel's samples.

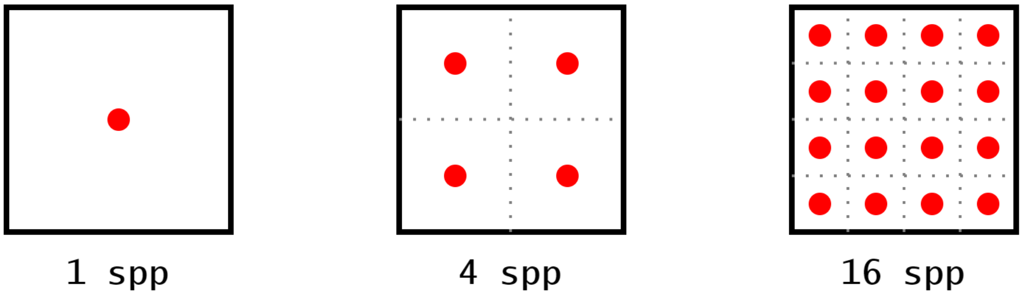



Figure 4.10: Multi-sampling grid

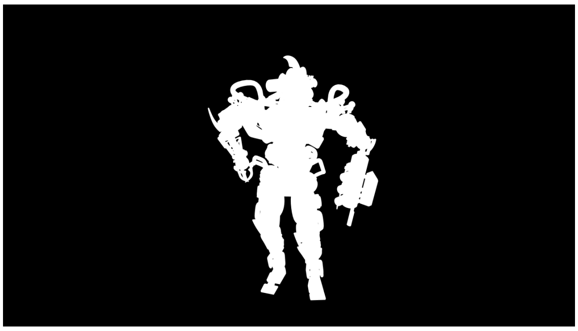
# 5 Results

## 5.1 Pipelines

### 5.1.1 Output

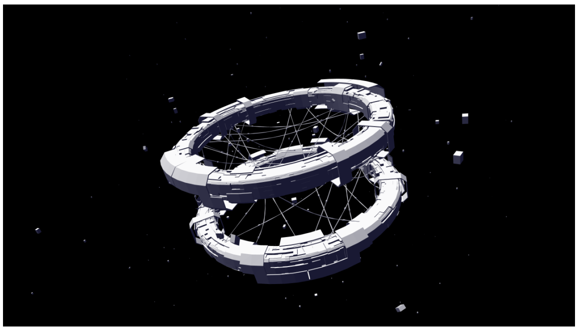
(a) Standard pipeline

(b) Shadow pipeline

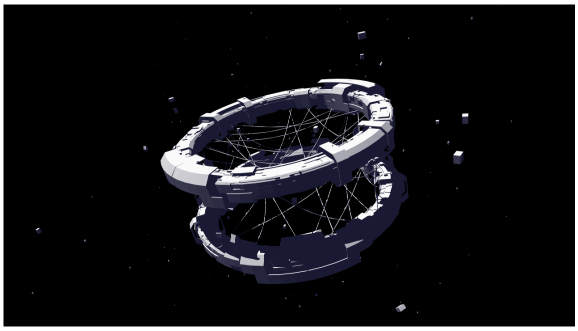


(c) Basic pipeline

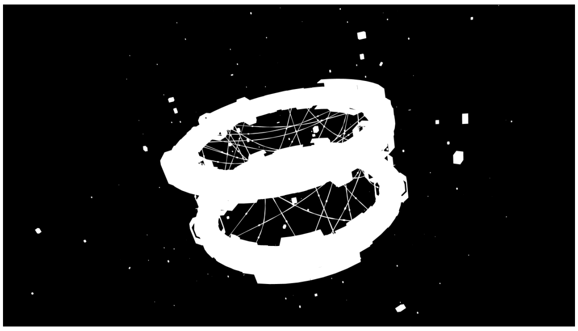
Figure 5.1: Output images using different pipelines for the scene "BrainStem"



(a) Standard pipeline

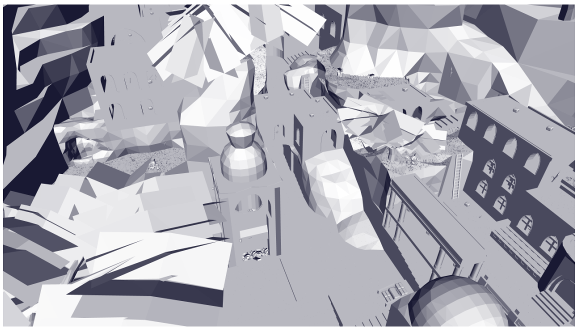


(b) Shadow pipeline

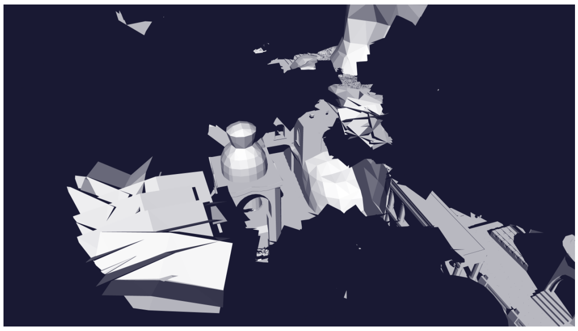


(c) Basic pipeline

Figure 5.2: Output images using different pipelines for the scene "Space Station"



(a) Standard pipeline



(b) Shadow pipeline

Figure 5.3: Output images using different pipelines for the scene "Eternal Valley"

### 5.1.2 Performace

We tested the different pipelines for the scene "Eternal Valley". We generated 64 frames with a resolution of $1280 \times 720$ pixels and 64 samples per pixel. We also rebuilt the acceleration structure every frame and preferred a fast trace for optimal ray tracing durations.

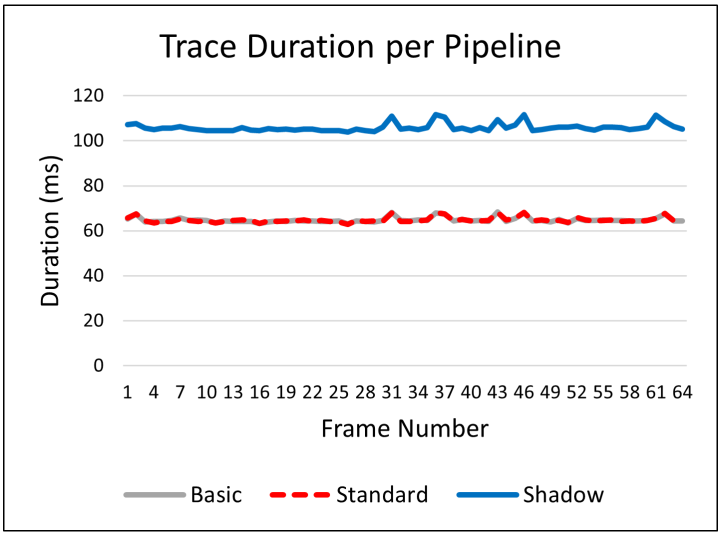



Figure 5.4: Impact of different pipelines on ray tracing duration

| Pipeline | Avg. Trace |
|---|---|
| Basic | 64.803 ms |
| Standard | 64.802 ms |
| Shadow | 105.898 ms |

Table 5.1: Average trace duration per pipeline

The basic pipeline and standard pipeline have basically the same behaviour. This would mean that our shaders do not significantly impact the performance of our pipelines and that the large majority of time is spent on the traversal of the acceleration structure.

We can also see that our shadow pipeline takes up much more time than the other two. This makes sense because it casts an additional secondary ray for every primary ray that hits a surface. In our test scene, every ray hits a surface (see Figure 5.1.1), which means that every primary ray also produces a shadow ray, and the total number of rays in the shadow pipeline is double the number of rays in the other two pipelines. However, the trace duration only increases by about 63% instead of 100%. This has probably to do with the way we cast our shadow rays, because we use two flags that are not used for primary rays:

- `gl_RayFlagsSkipClosestHitShaderEXT`

  This flag prevents the ray from invoking the closest hit shader. We can use it because our implementation relies entirely on the miss shader. The surface point is assumed to be in shadow (`shadow = true`) and the miss shader then sets `shadow` to `false`.

- `gl_RayFlagsTerminateOnFirstHitEXT`

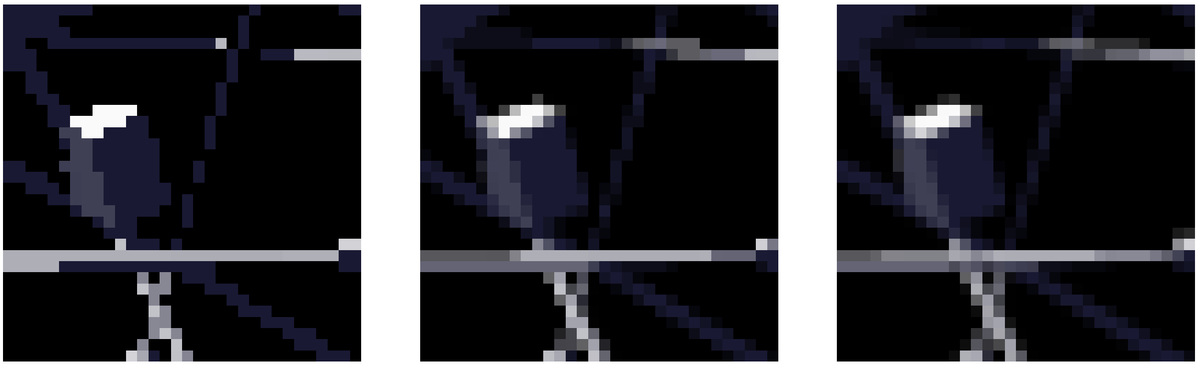
  This flag terminates the traversal of the acceleration structure as soon as a ray hits anything. We only use the miss shader, which means that as soon as we hit something, the miss shader will not be invoked, and we can safely terminate early.

This could explain why shadow rays are cheaper in our implementation.

Another possible reason is that all shadow rays are parallel because we use a directional light source. This makes them more coherent and may also improve performance.

## 5.2 Multi-Sampling

### 5.2.1 Output



(a) 1 spp (b) 4 spp (c) 16 spp

Figure 5.5: Comparison between images using a different number of samples per pixel

### 5.2.2 Performace

We tested different numbers of samples per pixel for the scene "Space Station" with the shadow pipeline. We generated 100 frames with a resolution of 1280 $\times$ 720 pixels. We also rebuilt the acceleration structure every frame and preferred a fast trace for optimal ray tracing durations.

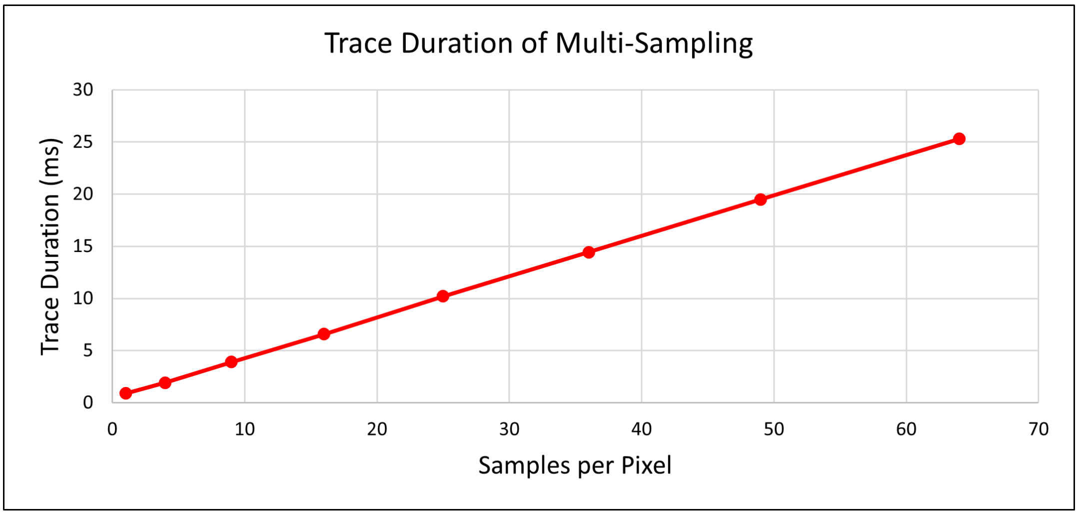


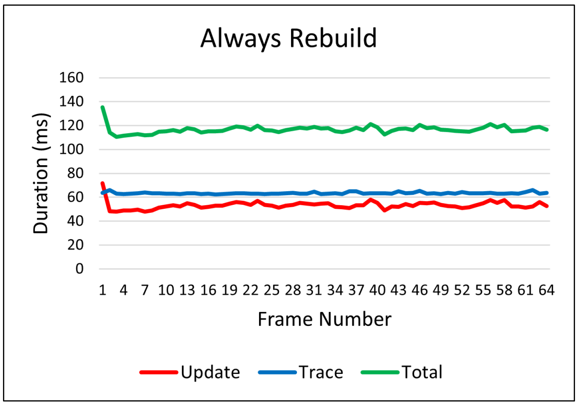

Figure 5.6: Impact of multi-sampling on ray tracing duration

This chart shows that the duration of tracing the rays scales linearly with the number of samples per pixel. This makes sense because if we double the number of samples, we also double the number of rays.

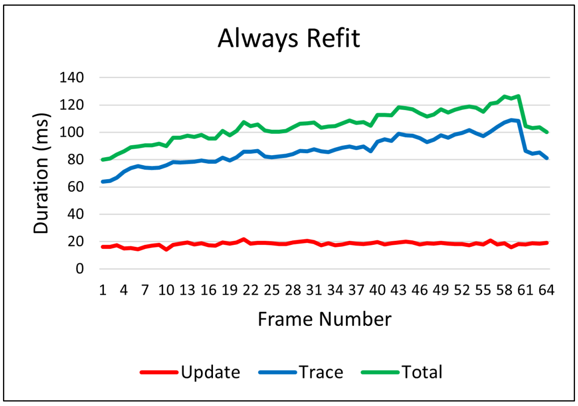
## 5.3 Update Strategies

We tested different strategies for updating the acceleration structure for the scene "Eternal Valley" with the standard pipeline. We generated 64 frames with a resolution of $1280 \times 720$ pixels and 64 samples per pixel and preferred a fast trace. We tested three strategies:

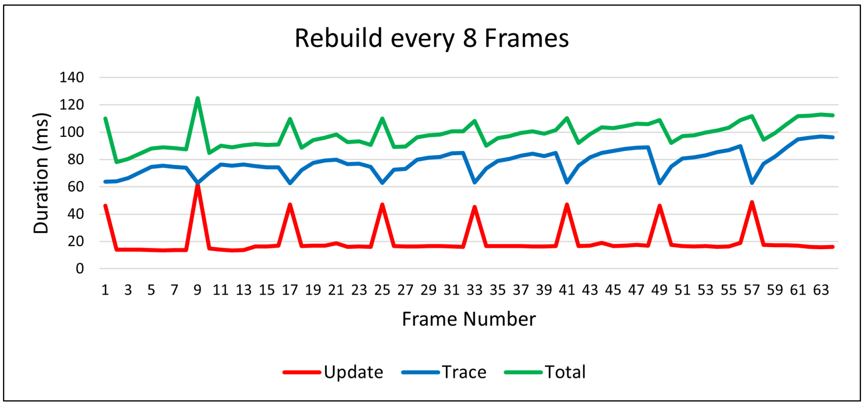
- **Always Rebuild**: The acceleration structure gets completely rebuilt every frame.
- **Always Rrefit**: After initially building the acceleration, we only refit it every frame.
- **Rebuild Every Eight Frames**: We refit most frames, but we do a rebuild every eighth frame.




(a) Always rebuild




(b) Always refit




(c) Rebuild every eight frames

Figure 5.7: Comparing different strategies for updating the acceleration structure

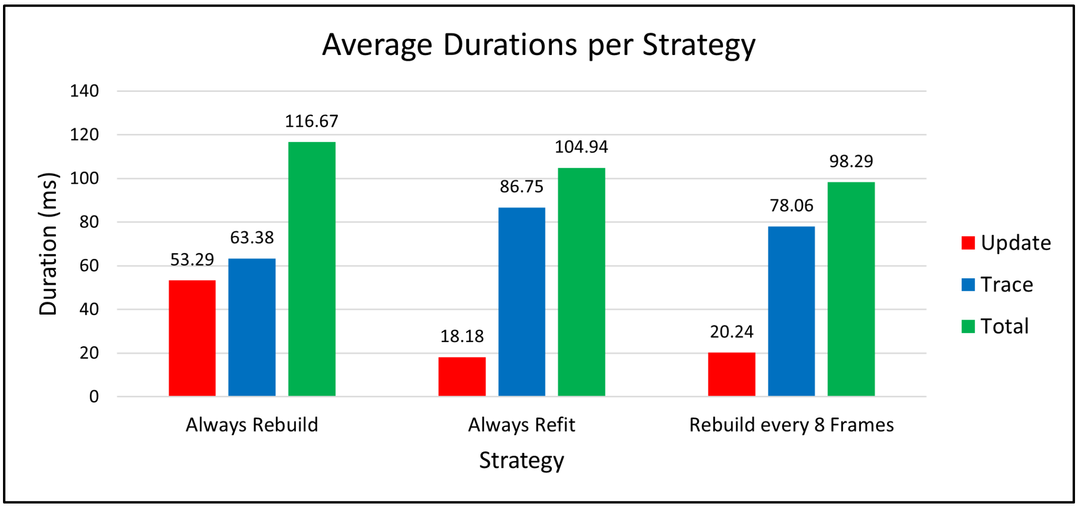



Figure 5.8: Comparing the average durations between different update strategies

### 5.3.1 Always Rebuild

As expected, always rebuilding gives us the shortest ray tracing duration, but it comes at the cost of a lot slower updates to the acceleration structure. The update and trace durations are very consistent; therefore, the total duration stays consistent. However, compared to the other strategies, it has the highest total duration on average.

### 5.3.2 Always Refit

This strategy results in very consistent and very short update durations. However, tracing the rays through the scenes becomes slower and slower, since the acceleration structure gets increasingly suboptimal. This causes the strategy to have a worse total time than always rebuilding towards the end of the animation. We can also notice that the trace duration suddenly decreases again at the end of the chart. The scene's animation is about 6 seconds long, and the delta time used was 0.1s. Thus, the animation finishes around frame 60 (shortly before the end of our chart) and starts to play again from the beginning. Since the BVH was also created at the beginning of the animation, refitting produces good results again.

### 5.3.3 Rebuild Every Eight Frames

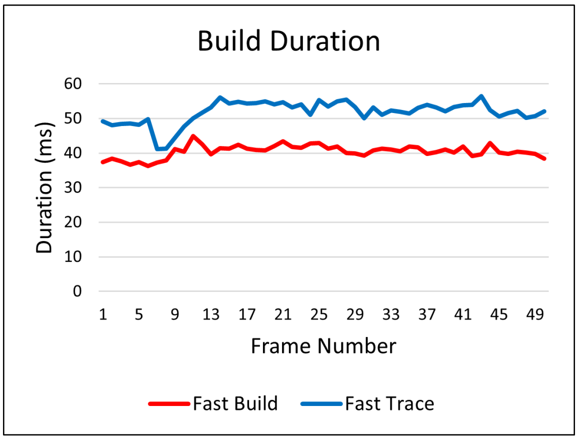
This strategy is a mix of the previous two. The update duration is usually very short, but every eight frames it spikes due to a rebuild occuring, which also causes the trace duration drops at those points. After each rebuild, the trace duration rises again, but consistent rebuilds prevent it from diverging too far from the initial duration. This strategy has good average update and trace durations, which result in the best total duration on average compared to the other strategies.

### 5.3.4 Discussion

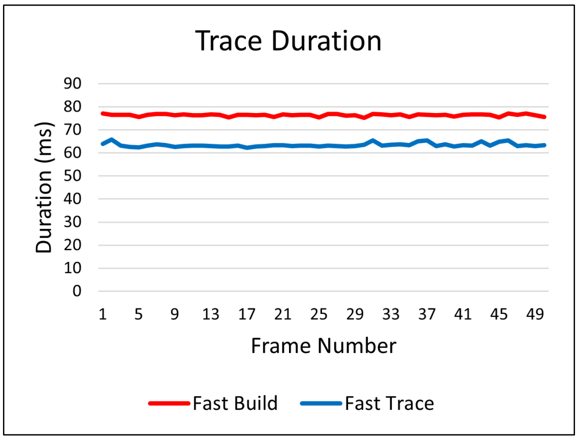
Because of the presented measurements, someone might assume that the third strategy is the best, but our measurements can not be generalized. If, for example, we keep increasing the number of rays, sooner or later, the strategy of always rebuilding will become more favorable. Always refitting, however, is rarely a good strategy. Even if not that many rays are traced, the BVH will become increasingly suboptimal in a lot of scenarios, which makes always refitting an unsustainable strategy. However, it might be viable for scenes that never change too much from their initial state.

## 5.4 Fast Build vs. Fast Trace

We tested the flags `PREFER_FAST_BUILD` and `PREFER_FAST_TRACE` for the scene "Eternal Valley" with the standard pipeline. We generated 50 frames with a resolution of $1280 \times 720$ pixels and 64 samples per pixel. We also rebuilt the acceleration structure for every frame since these flags do not affect refitting.




(a) Build duration




(b) Trace duration

Figure 5.9: Comparing the build and trace duration of the flag `PREFER_FAST_BUILD` to the flag `PREFER_FAST_TRACE`

.

We can see the expected outcome of `PREFER_FAST_BUILD` having shorter build durations and longer trace durations compared to `PREFER_FAST_TRACE`. It is important to mention that these flags do not impact refitting. These flags only affect the structure of the BVH, which refitting does not touch.

Whether `PREFER_FAST_BUILD` or `PREFER_FAST_TRACE` should be used depends on the specific scenario. If many rays are traced and the trace duration is the bottleneck in the application, `PREFER_FAST_BUILD` should probably be used. On the other hand, if most of the time is spent on rebuilding (not refitting!), `PREFER_FAST_TRACE` is likely the better choice.

# 6 Conclusion

This thesis aimed to understand how hardware-accelerated ray tracing can be implemented using Vulkan. We explored how acceleration structures, the ray tracing pipeline, and the shader binding table work together to enable hardware-accelerated ray tracing. We also looked at how a mesh can be represented by a bottom-level acceleration structure, and how a node in a scene graph that uses a mesh can be represented as an instance in the top-level acceleration structure. This approach means we do not have to update the entire acceleration structure; instead, we only need to update the meshes that change by rebuilding or refitting their BLAS, and we may need to update the TLAS if the transforms of some nodes have changed.

We also explored how acceleration structures can be updated either by completely rebuilding them or by only refitting the volumes of the BVH to the new positions. In our testing, a combination of both methods, where we refit most of the time and periodically rebuild, proved to be the most efficient in terms of average total rendering time. In many scenarios, a variation of this strategy is likely the best approach. However, rendering duration can vary significantly from frame to frame, and a rebuild can cause visible spikes in rendering time. In a real-time application, we would need a way to smooth out these spikes, for example, by rebuilding different meshes across different frames, which is possible thanks to the two-level hierarchy of the acceleration structure. We also have the option to rebuild less dynamic meshes less frequently and more dynamic ones more often, which may further improve performance. This strategy may not always be ideal. If a large number of rays are cast in the application, it may be better to rebuild the acceleration structures every frame to optimize tracing performance. In these situations, setting the `PREFER_FAST_TRACE` flag often makes sense.

However, casting only a few rays relative to the number of dynamic objects does not automatically justify refitting every frame, because refitting is not sustainable in many scenes. Refitting only gives good results if the scene does not move too far away from its original state. If the scene keeps changing significantly, rendering time will steadily increase.

Finally, we also saw that not all rays have equal cost. If possible, we can use flags like `gl_RayFlagsSkipClosestHitShaderEXT` and `gl_RayFlagsTerminateOnFirstHitEXT`, which can significantly reduce ray cost. As we observed, our shadow rays were about 37% faster than regular rays.

# List of Figures

# List of Tables

# Bibliography

[App68] A. Appel. "Some techniques for shading machine renderings of solids". In: *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*. AFIPS '68 (Spring). Atlantic City, New Jersey: Association for Computing Machinery, 1968, pp. 37–45. ISBN: 9781450378970. URL: https://doi.org/10.1145/1468075.1468082.

[Chr+18] P. Christensen, J. Fong, J. Shade, W. Wooten, B. Schubert, A. Kensler, S. Friedman, C. Kilpatrick, C. Ramshaw, M. Bannister, B. Rayner, J. Brouillat, and M. Liani. "An Advanced Path Tracing Architecture for Movie Rendering". In: *ACM Transactions on Graphics*. Vol. 37. 3. 2018. URL: https://docslib.org/doc/639313/an-advanced-path-tracing-architecture-for-movie-rendering.

[Cor18] N. Corporation. *Turing Architecture White Paper*. Tech. rep. NVIDIA Corporation, 2018.

[GPU] GPUOpen-LibrariesAndSDKs. *Vulkan Memory Allocator*. URL: https://github.com/GPUOpen-LibrariesAndSDKs/VulkanMemoryAllocator.git (visited on 09/17/2025).

[HVH16] C. Hery, R. Villemin, and F. Hecht. *Towards Bidirectional Path Tracing at Pixar*. Tech. rep. 2016. URL: https://graphics.pixar.com/library/BiDir/paper.pdf.

[Hue+25] R. Huerta, M. A. Shoushtary, J.-L. Cruz, and A. González. *Analyzing Modern NVIDIA GPU cores*. 2025. arXiv: 2503.20481 [cs.AR]. URL: https://arxiv.org/abs/2503.20481.

[Jac+12] A. Jacobson, I. Baran, L. Kavan, J. Popović, and O. Sorkine. "Fast Automatic Skinning Transformations". In: *ACM Transactions on Graphics (proceedings of ACM SIGGRAPH)* 31.4 (2012), 77:1–77:10.

[KA13] T. Karras and T. Aila. "Fast parallel construction of high-quality bounding volume hierarchies". In: *Proceedings of the 5th High-Performance Graphics Conference*. HPG '13. Anaheim, California: Association for Computing Machinery, 2013, pp. 89–99. ISBN: 9781450321358. DOI: 10.1145/2492045.2492055. URL: https://doi.org/10.1145/2492045.2492055.

[Kav+07] L. Kavan, S. Collins, J. Žára, and C. O'Sullivan. "Skinning with dual quaternions". In: *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games*. I3D '07. Seattle, Washington: Association for Computing Machinery, 2007, pp. 39–46. ISBN: 9781595936288. DOI: 10.1145/1230100.1230107. URL: https://doi.org/10.1145/1230100.1230107.

[KŽ05] L. Kavan and J. Žára. "Spherical blend skinning: a real-time deformation of articulated models". In: *Proceedings of the 2005 Symposium on Interactive 3D Graphics and Games*. I3D '05. Washington, District of Columbia: Association for Computing Machinery, 2005, pp. 9–16. ISBN: 1595930132. DOI: 10.1145/1053427.1053429. URL: https://doi.org/10.1145/1053427.1053429.

[Khr21] Khronos Group. *glTF™ 2.0 Specification*. The Khronos Group Inc. 2021. URL: https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html (visited on 09/22/2025).

[Khr25] Khronos Group. *Vulkan® 1.4.327 - A Specification (with all registered Vulkan extensions)*. The Khronos Group Inc. 2025. Chap. 39. Acceleration Structures. URL: https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html#acceleration-structure (visited on 09/22/2025).

[Koc+20] D. Koch, T. Hector, J. Barczak, and E. Werness. *Ray Tracing in Vulkan*. 2020. URL: https://www.khronos.org/blog/ray-tracing-in-vulkan (visited on 09/25/2025).

[Kop+12] D. Kopta, T. Ize, J. Spjut, E. Brunvand, A. Davis, and A. Kensler. "Fast, effective BVH updates for animated scenes". In: *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. I3D '12. Costa Mesa, California: Association for Computing Machinery, 2012, pp. 197–204. ISBN: 9781450311946. DOI: 10.1145/2159616.2159649. URL: https://doi.org/10.1145/2159616.2159649.

[Lau+09] C. Lauterbach, M. Garland, S. Sengupta, D. Luebke, and D. Manocha. "Fast BVH construction on gpus". In: *Computer Graphics Forum* 28 (Apr. 2009), pp. 375–384. DOI: 10.1111/j.1467-8659.2009.01377.x.

[Lun] C. Lunarg. *vk-bootstrap: Vulkan Bootstrapping Library*. URL: https://github.com/charles-lunarg/vk-bootstrap (visited on 09/19/2025).

[MT05] T. Möller and B. Trumbore. "Fast, minimum storage ray/triangle intersection". In: *ACM SIGGRAPH 2005 Courses*. SIGGRAPH '05. Los Angeles, California: Association for Computing Machinery, 2005, 7–es. ISBN: 9781450378338. DOI: 10.1145/1198555.1198746. URL: https://doi.org/10.1145/1198555.1198746.

[PL10] J. Pantaleoni and D. Luebke. "HLBVH: hierarchical LBVH construction for real-time ray tracing of dynamic geometry". In: *Proceedings of the Conference on High Performance Graphics*. HPG '10. Saarbrucken, Germany: Eurographics Association, 2010, pp. 87–95.

[Pop+06] S. Popov, J. Gunther, H.-p. Seidel, and P. Slusallek. "Experiences with Streaming Construction of SAH KD-Trees". In: *2006 IEEE Symposium on Interactive Ray Tracing*. 2006, pp. 89–94. DOI: 10.1109/RT.2006.280219.

[Sha05] Y. Shafranovich. *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. RFC 4180. Oct. 2005. DOI: 10.17487/RFC4180. URL: https://www.rfc-editor.org/info/rfc4180 (visited on 09/15/2025).

[SK07] A. Soupikov and A. Kapustin. "Highly Parallel Fast KD-tree Construction for Interactive Ray Tracing of Dynamic Scenes". In: *Comput. Graph. Forum* 26 (Sept. 2007), pp. 395–404. DOI: `10.1111/j.1467-8659.2007.01062.x`.

[Wal07] I. Wald. "On fast Construction of SAH-based Bounding Volume Hierarchies". In: *2007 IEEE Symposium on Interactive Ray Tracing*. 2007, pp. 33–40. DOI: `10.1109/RT.2007.4342588`.

[WBS07] I. Wald, S. Boulos, and P. Shirley. "Ray tracing deformable scenes using dynamic bounding volume hierarchies". In: *ACM Trans. Graph.* 26.1 (Jan. 2007), 6–es. ISSN: 0730-0301. DOI: `10.1145/1189762.1206075`. URL: `https://doi.org/10.1145/1189762.1206075`.

[Wal+14] I. Wald, S. Woop, C. Benthin, G. S. Johnson, and M. Ernst. "Embree: a kernel framework for efficient CPU ray tracing". In: 33.4 (July 2014). ISSN: 0730-0301. URL: `https://doi.org/10.1145/2601097.2601199`.

[Whi80] T. Whitted. "An improved illumination model for shaded display". In: *Commun. ACM* 23.6 (June 1980), pp. 343–349. ISSN: 0001-0782. DOI: `10.1145/358876.358882`. URL: `https://doi.org/10.1145/358876.358882`.

[Wil+05] A. Williams, S. Barrus, R. K. Morley, and P. Shirley. "An efficient and robust ray-box intersection algorithm". In: *ACM SIGGRAPH 2005 Courses*. SIGGRAPH '05. Los Angeles, California: Association for Computing Machinery, 2005, 9–es. ISBN: 9781450378338. DOI: `10.1145/1198555.1198748`. URL: `https://doi.org/10.1145/1198555.1198748`.

[zeu] zeux. *volk*. URL: `https://github.com/zeux/volk.git` (visited on 09/17/2025).